UNIT 01 · CHAPTER 4

Tokenization

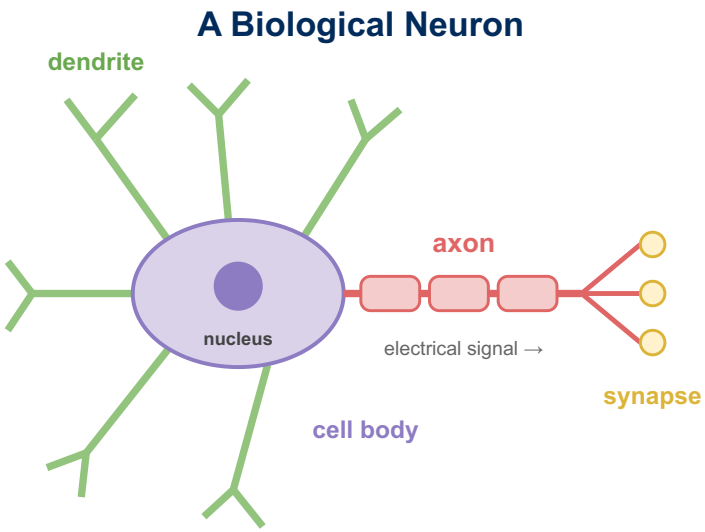
CSE 10124 · Building ChatGPT

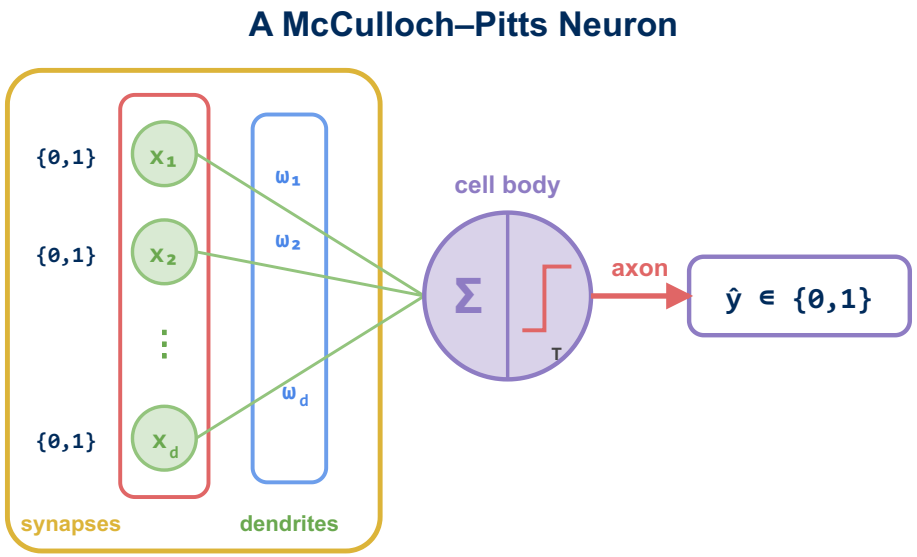
William Theisen · University of Notre Dame



Activation Functions

Activation Function: a function applied to a neuron's weighted sum that determines the neuron's output.





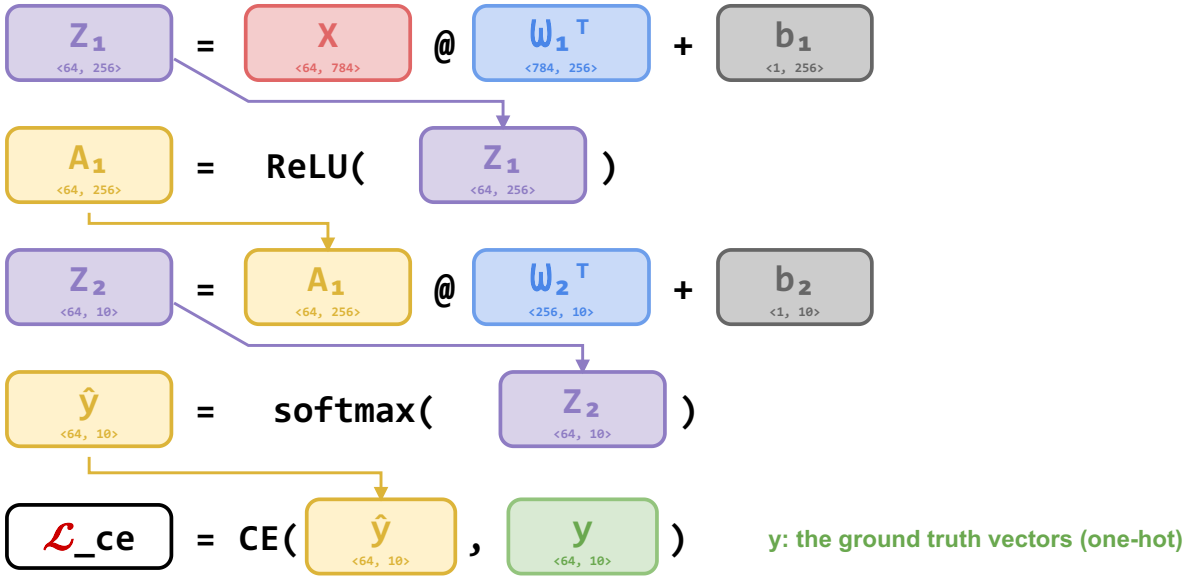
$$z = \omega_1 x_1 + \omega_2 x_2 + \cdots + b$$

$$\underset{\langle 1,\,10\rangle}{Z} = \underset{\langle 1,\,784\rangle}{X} @ \underset{\langle 784,\,10\rangle}{W^T} + \underset{\langle 1,\,10\rangle}{b}$$

Linear Transformation

Computation Graph with Activation Functions

Essentially, we just add an **activation function** **after every linear layer**.



Computation Graph with Activation Functions

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class FFN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 256)
        self.fc2 = nn.Linear(256, 10)

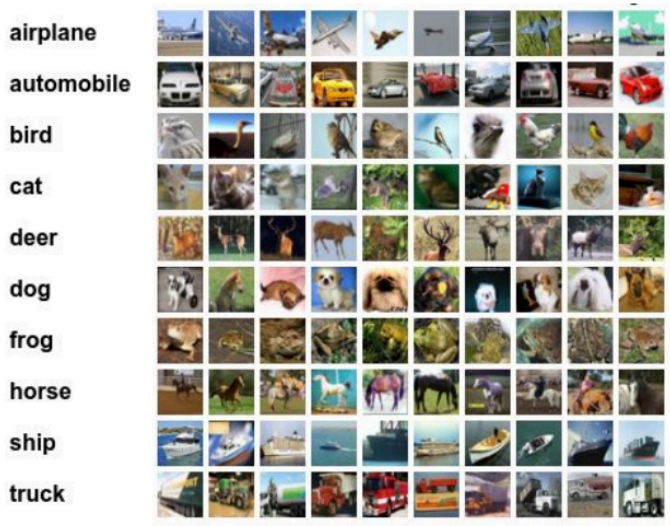
    def forward(self, x):
        x = F.relu(self.fc1(x))
        logits = self.fc2(x)
        return logits

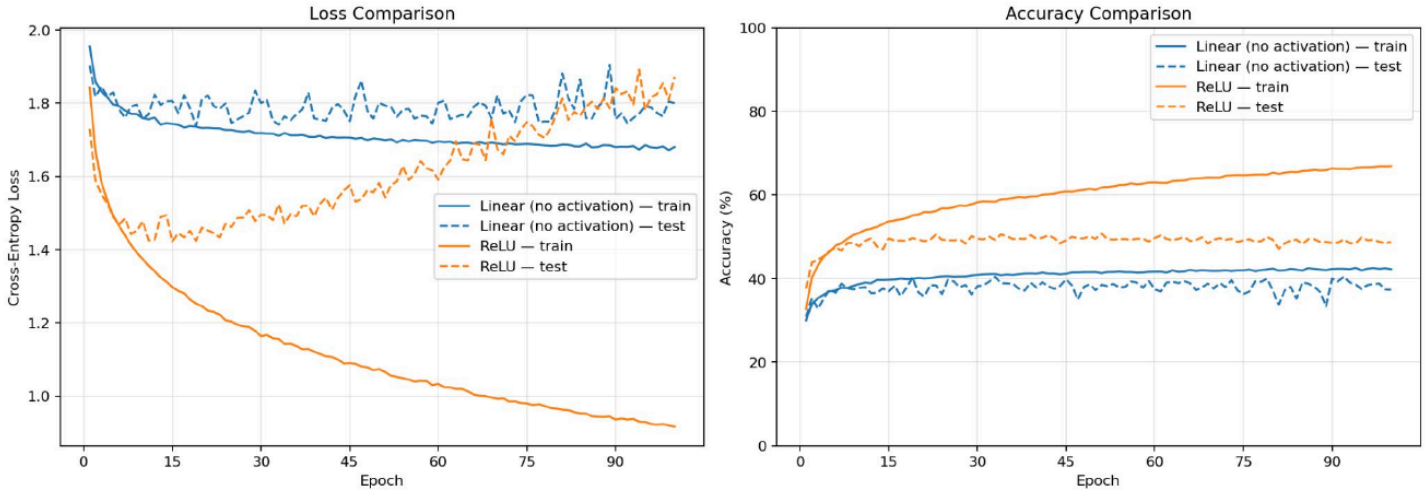
X = torch.rand(64, 28 * 28)   # a batch of 64 images
logits = FFN()(X)
print(logits.shape)
print(F.softmax(logits, dim=1).shape)
```

OUTPUT

```
torch.Size([64, 10])
torch.Size([64, 10])
```

Classification Dataset: CIFAR-10

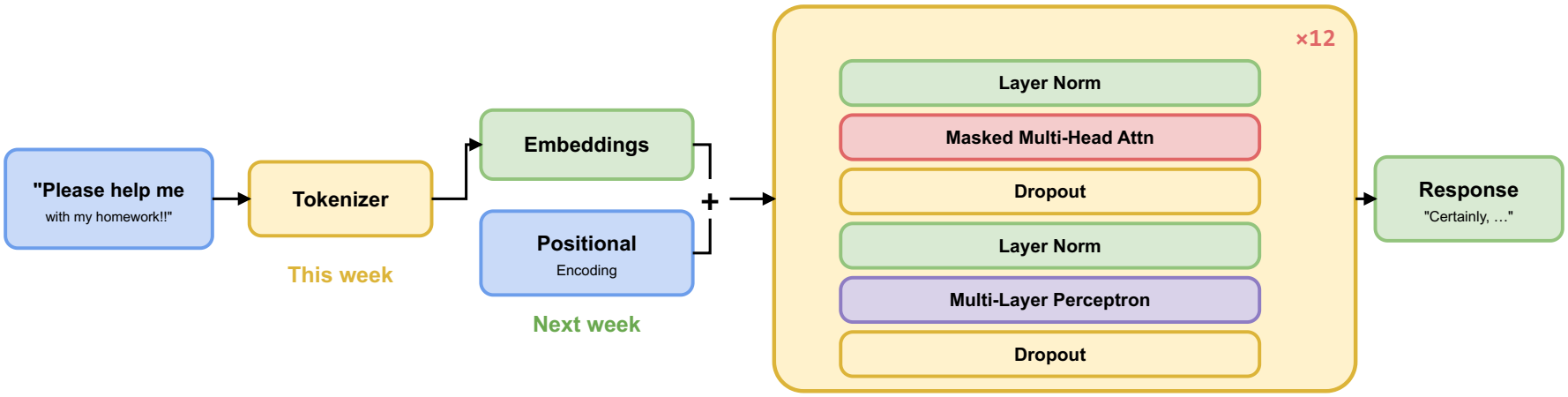




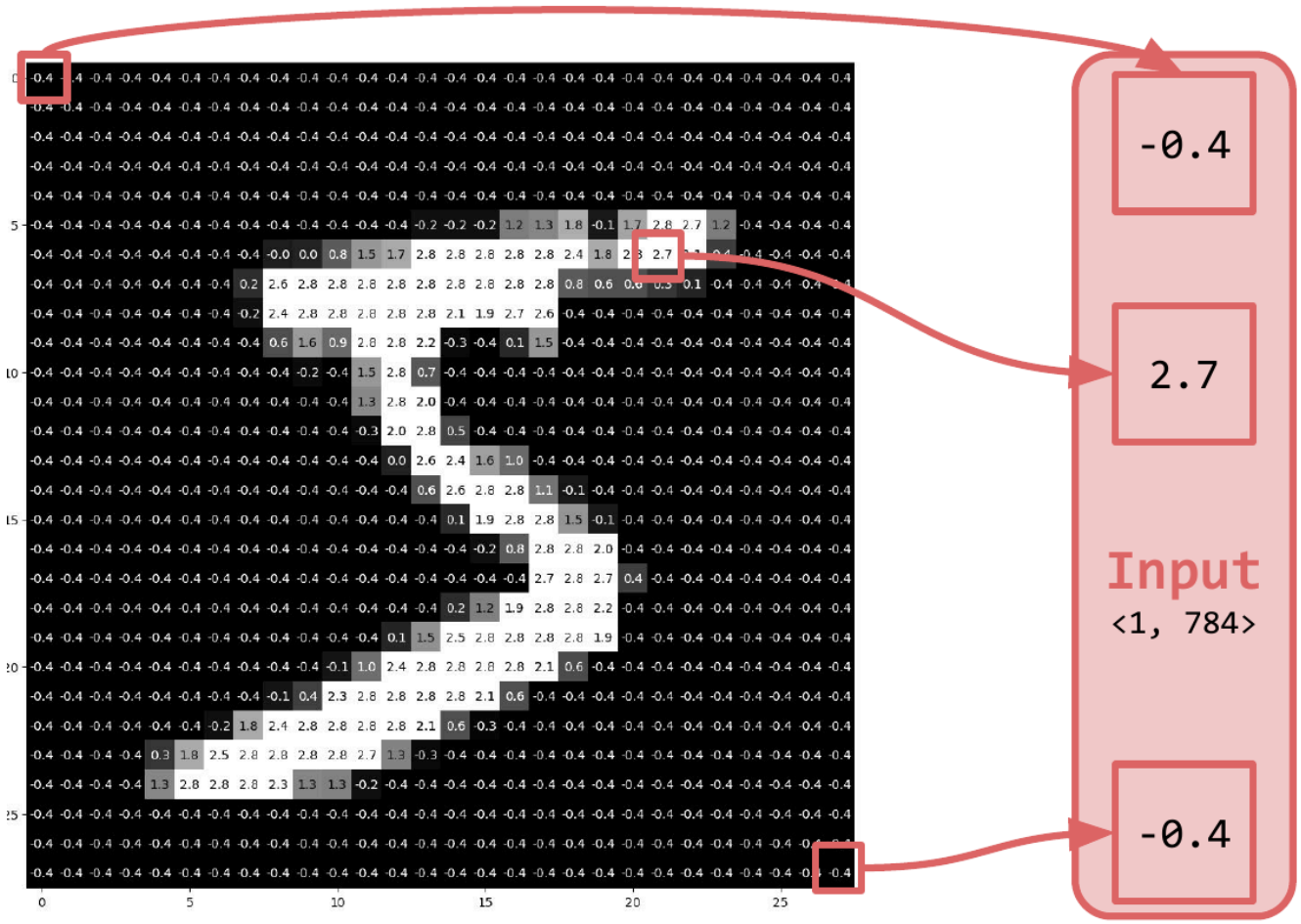
CSE 10124: Building ChatGPT

Last week we built a simple **neural network**. This is one of the pieces we need to build **GPT-2**.

Over the next two weeks, we'll build the pieces we need to actually get **input** into the model.



Representing Image Input



Our images are **<28, 28>** so we can **flatten** them into a **<1, 784>** dimensional **vector**.

Representing Text Input

Input Prompt

`"The cat sat on the mat and ate."`

Token: one piece of text represented by an entry in a model's **vocabulary**.



Vocabulary: the fixed collection of token pieces a tokenizer can represent.



Tokenization

Tokenization: the procedure that splits text into pieces and maps those pieces to **token IDs**. **Token ID**: an integer label assigned to a vocabulary entry.



Tokenizer's Vocabulary

Token	Token ID
"."	0
"The"	1
" and"	2
" ate"	3
" cat"	4
" mat"	5
" on"	6
" sat"	7
" the"	8

Encoding: converting text into **token IDs**.



Decoding

Decoding: reconstructing text from **token IDs**.



Token	Token ID
"."	0
"The"	1
" and"	2
" ate"	3
" cat"	4
" mat"	5
" on"	6
" sat"	7
" the"	8

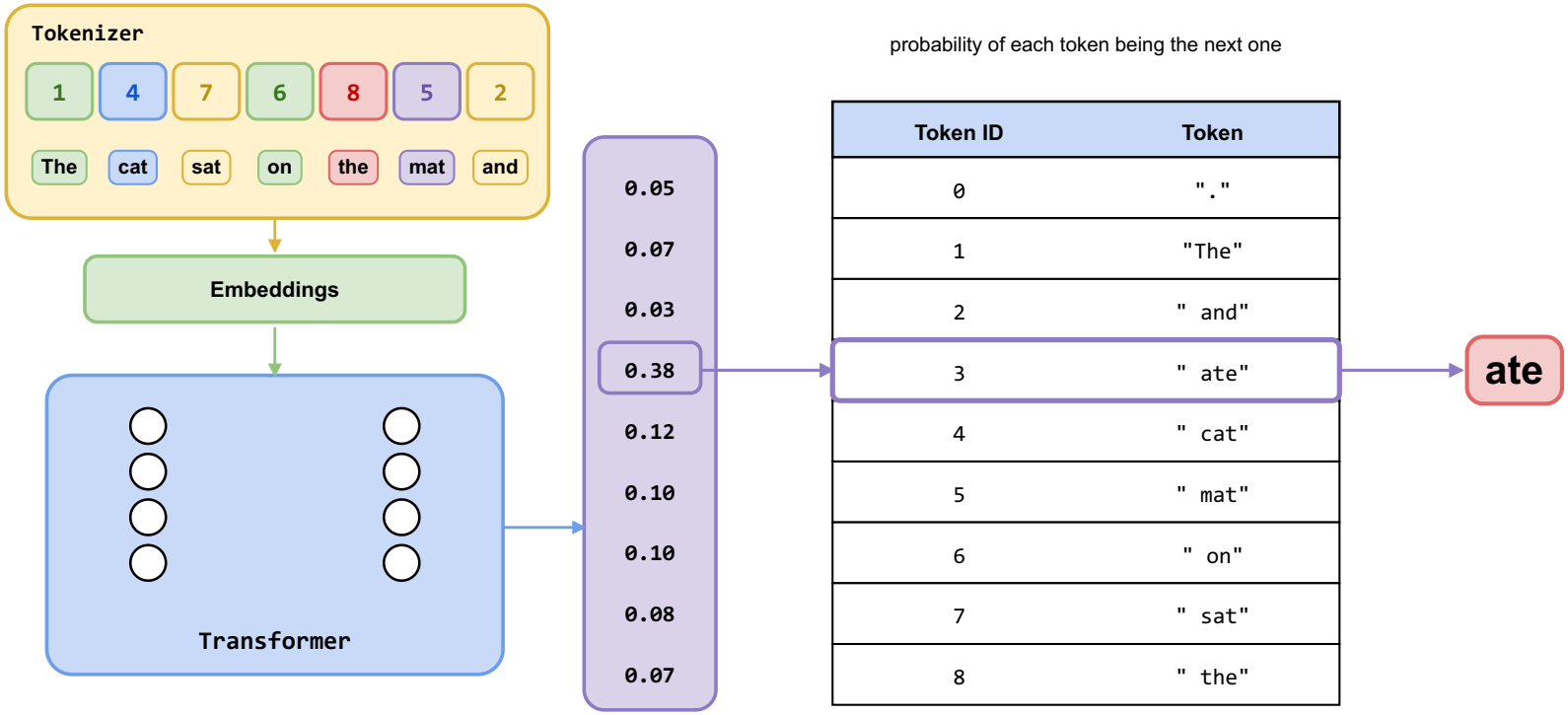
The **token IDs** capture no intrinsic meaning.

Comparing IDs does not measure meaning.

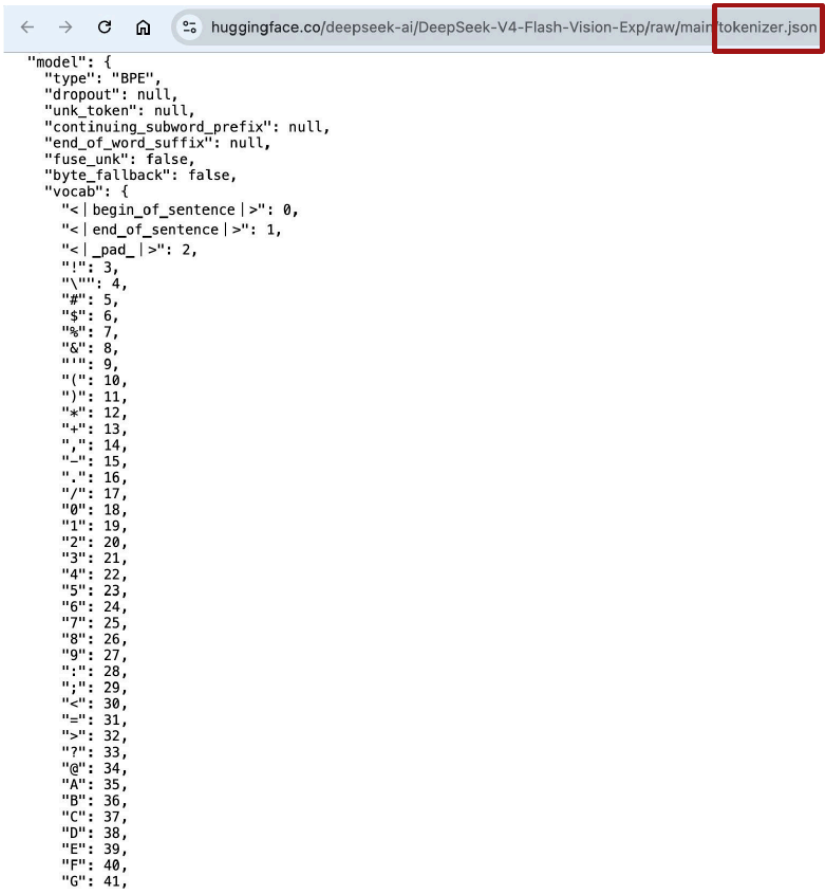


Output of an LLM

The output of an **LLM** is a probability for every **token** in the **vocabulary** being the next one.



Tokenization in LLMs



JSON: **J**ava**S**cript **O**bject **N**otation

Different Tokenization Levels

Input Prompt

"The cat sat on the mat and ate."

Word Level

The | cat | sat | on | the | mat | and | ate | .

Character Level

T h e ␣ c a t ␣ s a t ␣ o n ␣ t h e ␣ m a t ␣

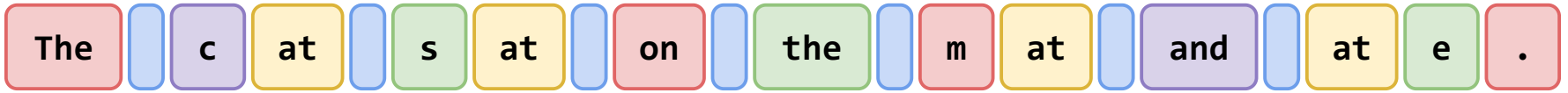
a n d ␣ a t e .

Notice the Repeated "at"

Word Level



Subword: a reusable piece smaller than a whole word.



Subwords in the Brain

Rastle, Davis and New (*Psychonomic Bulletin & Review*, 2004): a word flashed too quickly to notice makes people faster at reading a second word, when the first looks like it's **built from the second**.

flashed	then read	related by	faster by
`cleaner`	`CLEAN`	meaning and pieces	27 ms
`corner`	`CORN`	pieces only (corn + er)	22 ms
`brothel`	`BROTH`	letters only ("el" isn't an ending)	4 ms

Readers split words into pieces by spelling, not meaning. irishQwen's tokenizer does too:

- `·cleaner` → `·c le an er`, `·corner` → `·c or n er`
- `·brothel` → `·b r ot he l` (no `oth`, unlike `·broth` → `·b r oth`)

Different pieces though: readers know real endings like "-er", BPE only knows what was common in Shakespeare.

OpenAI Tokenizer

platform.openai.com/tokenizer

